

On the Impact of Pruning on Model Robustness

Vilmos Vörös, HW921K

May 31, 2026

Contents

1	Background	2
1.1	Global Magnitude-based Pruning	3
1.2	Out-of-Distribution and Adversarial Robustness	3
2	Methodology	4
2.1	CIFAR Corrupted Benchmark	4
2.2	Fast Adaptive Boundary	4
2.3	Fourier Sensitivity Method	5
3	Experiments	6
3.1	Sparsity-Robustness tradeoff	6
3.2	Sensitivity analysis	9
4	Summary	10
A	Appendix	12

Introduction

In numerous fields of industry - such as healthcare, computer vision, natural language processing, etc. - neural networks have been deployed to facilitate various applications that were not possible before [1]. However, the requirements for a deployable model are manifold. Apart from being accurate enough to yield economic value, the model also needs to be energy efficient and adaptable to hardware constraints, while being robust to maliciously altered inputs and changes in the input distribution.

Our work focuses on a custom pruning strategy and examines its impact from a robustness perspective. By trying to examine robustness from different angles, we aim to improve our understanding of the relationship between model sparsity and robustness and support our future work on improving our pruning strategy.

After presenting the motivations and various definitions of robustness in chapter 1, we discuss the resources and algorithms we used for our experiments in chapter 2. The results of our experiments are presented in chapter 3, followed by our summary and future research directions.

1 Background

Large neural networks have produced state-of-the-art results in numerous fields such as computer vision or natural language processing. To deploy these neural network model in an industry application, the most common option is training the model in a data center utilizing huge GPU clusters, liquid cooling, etc. At inference time, the clients could also access the model in the data center via internet.

However, with the proliferation of Internet-of-Things (IoT) devices and applications, neural network models are required to run more efficiently on resource-constrained devices to facilitate the growing number of edge AI use cases [2]. Efficiency or compactness refers to the model giving outputs accurate enough to enable the specific application, while also inducing low memory and energy-consumption overhead. The latter two translates to the model having a small parameter count and an architecture which minimizes latency and floating-point operations (FLOPs). Depending on the deployment scenario, a model may receive inputs that are far from the distribution of the training data or inputs that are manipulated to make the model produce a wrong output. Robustness of the model refers to its ability to maintain its performance even on such inputs. To sum up, a trained neural network has to satisfy accuracy, compactness and robustness requirements in order to become qualified for industrial applications.

In academia, there has been extensive research to improve them, but most works tackled them separately [1] [3]. To improve accuracy, the general approach was training overparameterized networks with regularization. Model compression techniques (i.e. weight pruning, knowledge distillation) and novel architecture have been proposed to achieve compactness while retaining accuracy. Finally, increasing robustness has been achieved by augmenting the training dataset to better represent the distribution of inputs at inference [4].

However, meeting these requirements simultaneously - so not increasing one at the expense of another - was found to be a challenge [5]. Therefore, it is important to evaluate a

technique tackling one of the three requirements on all three aspects. In my case, measuring the impact of model compression on accuracy and robustness as well. In the following sections, we give an overview of the techniques relevant to this work.

1.1 Global Magnitude-based Pruning

There are many contemporary methods for model compression. The main four categories include network pruning, knowledge distillation, tensor decomposition, and quantization [6], although it is also common to propose hybrid strategies. Network pruning refers to evaluating weights or groups of weights of the model based on a certain importance criteria and removing the ones which were considered the least important. This importance signal could be weight magnitude, the sensitivity of loss with respect to the weight, or other heuristics. After pruning, a fine-tuning or retraining phase is applied using the remaining weights in order to refit the pruned model to the training data. When the magnitude is utilized for representing the importance of weights, the pruning step uses a threshold for comparison, and every weight with magnitude below the threshold gets removed. Global pruning refers to having only one threshold and using it across all layers, regardless of their shape and function (i.e. classifier layer, convolutional layer). If every layer has its separate, local threshold, the pruning is applied uniformly across the entire network, which can lead to sub-optimal sparsity and the increased complexity of tuning these thresholds for a deep network [3]. In contrast, global pruning enables layers with different roles (i.e. feature extractor, classifier) to contribute to the overall sparsity non-uniformly.

1.2 Out-of-Distribution and Adversarial Robustness

A basic assumption in traditional machine learning theory is that training and test samples follow the same distribution. However, in many applications this assumption may be violated due to distributional shifts in the test set. Although the degraded performance due to the distributional shifts may not be critical in recommendation or image classification systems, decreased accuracy in healthcare or self-driving cars could even result in fatalities. Therefore, the Out-of-Distribution (OOD) Generalization Problem has been formulated and an increasing number of methods and evaluation procedures have been developed accordingly [7]. Popular categorization distinguishes shifts as either covariate or concept shifts. The former refers to change in the input features distribution $P(x)$, while the latter indicates a different underlying concept or label mapping $P(Y|X)$ present compared to the one generating the training samples.

Besides OOD inputs, an application-ready model is required to be robust to so-called adversarial inputs as well. By definition, adversarial inputs are maliciously altered data points of the original distribution which are used to make the model generate incorrect outputs [8]. As the name implies, there are malevolent actors involved who apply sophisticated methods to hide these modifications from a human inspector. In case of vision models, various optimization strategies can be utilized to find a specific perturbation applied to the input image to achieve its misclassification. These strategies for corrupting in-distribution inputs are referred to as adversarial attacks, while adversarial training indicates certain modifications in the training process to increase robustness to such inputs.

2 Methodology

In this section I present the benchmark dataset and adversarial algorithm utilized to evaluate OOD and adversarial robustness of a (pruned) network.

2.1 CIFAR Corrupted Benchmark

The CIFAR-10 and CIFAR-100 datasets have been used as benchmarks for decades in the field of computer vision [9]. The two datasets consists of 60,000 32x32 RGB images with 10 and 100 different classes, respectively. The images are uniformly distributed across the classes and objects are at the center, thus it serves as a clean baseline without any modification that may happen in real-world scenarios.

To facilitate the prediction of the impact of common image corruptions the trained model, the CIFAR-10-C and CIFAR-100-C datasets were constructed [10]. The authors applied 19 different types of corruptions at varying intensities on the test sets of CIFAR-10 and CIFAR-100, respectively. More specifically, corruptions are categories into followig categories: noise, blur, weather conditions, and digital corruptions, while intensity levels span from 1 to 5. Figure 1 shows an example for every category, applied on the same image from CIFAR-10 dataset.



Figure 1: Different types of corruptions with severity level 5 on a CIFAR-10 32x32 RGB image

In the context of the OOD generalization problem mentioned in section 1.2, these real-world distortions cause a covariate shift in the distribution of the test data. In other words, only input features suffer a distributional shift, while the underlying concept - the class of photographed object in this case - remains the same.

2.2 Fast Adaptive Boundary

Apart from OOD robustness, a neural network may need be prepared for adversarial attacks, which were introduced in section 1.2. These adversarial attacks can be categorized in many ways. Methods which have full control over the network are called white-box attacks, while black-box attacks can only give an input and inspect the corresponding output of the model. In case of the former category, some algorithms are looking for the minimal perturbation on

the input image which achieves misclassification, while others return an altered image whose distance from the original is just thresholded.

Finding minimal perturbations for deep neural networks yields NP-hard optimization problems whose computational costs are prohibitive. Thus, finding scalable heuristics which find high-quality results fast became the focus of current literature. Regarding adversarial robustness of trained models, an attack that overestimates the minimal perturbation also overestimates the robustness of the model, thus encouraging false conclusions.

The Fast Adaptive Boundary (FAB) attack was specifically tailored to yield high-quality results fast with minimal hyperparameter-tuning required [11]. It uses a straight hyperplane to approximate the complex decision boundary between the true class and any other wrong class, utilizing raw output logits and their gradients to with respect to input pixels. After projecting the original image to the closest decision boundary, the targeted point is calculated via a convex combination of the projected and the original point to keep the final point close to the original, hence approximating the minimal perturbation better. Unlike the Projected Gradient Descent attack and many others which maximize the loss function by altering the input image, FAB algorithm uses the gradient of the difference between logit values corresponding to different classes. Thus, it is robust to so-called gradient masking where the loss gradient signal is obfuscated via random noise or non-differentiable operations which make the loss landscape jagged and un-smooth. In case of heavily pruned networks, the loss landscape has many flat zones, which makes the gradient vector sparse. A sparse gradient vector then hinders finding the decision boundary. In conclusion, FAB is ideal to measure the adversarial robustness of a sparse network based on the magnitude/norm of the minimal perturbation it finds. As the norm of this perturbation is a good proxy for the distance between decision boundaries, we can infer how stable or robust the model’s mapping is.

2.3 Fourier Sensitivity Method

The primary method to train a robust model is data augmentation, which includes generating out-of-distribution samples and inserting them into the training set. This requires a good understanding of what distortions or corruptions the model needs to be robust against in a real-world setting. In case of image classification, transformations like rotation, cropping or flipping are commonly used to inject the inductive bias of invariance to these transformations into the model. As seen in section 2.1, benchmarking the performance of models on different corruption types can provide a sense of robustness. However, these common corruptions may not cover all possible features of distortions in the input space.

To facilitate a better understanding of the impact of pruning on model robustness, I utilize the Fourier sensitivity method [12]. The method involves applying different Fourier basis functions to the input image and measuring the model’s error rate on the perturbed test set. The 2D basis matrices capture perturbations with different frequencies and orientations. By evaluating average error rates on perturbed sets corresponding to each basis matrix, the method outputs a sensitivity heatmap which visualizes how the error rate changes due to high-frequency or low-frequency perturbations, as well as horizontal or vertical ones.

In this way, we can examine how pruning alters the model’s sensitivity to more general perturbations, compared to the specific corruptions found in the CIFAR-10-C dataset.

3 Experiments

To evaluate the impact of gradual magnitude-based pruning on OOD and adversarial robustness, we conducted experiments using the ResNet20 architecture and the CIFAR-10 and CIFAR-10-C datasets. The datasets were presented in section 2.1.

The architecture of ResNet20 [13] is presented in Table 1. $C(n)$ denotes 2D convolutional layers with n output channels, BN denotes batch normalization layers, GAP represents a Global Average Pooling layer, while $FC(n)$ denotes a fully connected layer with n output units. To mitigate the vanishing gradient problem, the architecture includes residual connections in each residual block, which are denoted by square brackets.

$C(16)$ -BN, $[C(16)$ -BN, $C(16)$ -BN] $\times 3$, $[C(32)$ -BN, $C(32)$ -BN] $\times 3$, $[C(64)$ -BN, $C(64)$ -BN] $\times 3$, GAP, FC(10)

Table 1: Architecture of ResNet20

For gradual pruning, we applied a custom strategy which was inspired by the Iteratively Reweighted l_1 -minimization problem [14]. By updating the weights of the regularization term after each epoch, the regularization was more effective in pushing weights towards zero than common l_1 -regularization. The training process consisted of the following phases: a "warm-up" phase of 20 epochs without the sparsifying regularization; the "regularization" phase to drive weights towards zero while fitting the data; a pruning step where weights with magnitudes below a certain threshold were set to zero; finally a "re-fitting" phase where the remaining weights could be updated to refit the data without the constant pressure of the regularization term. As the process was gradual, these three phases were repeated until a sparsity level of 95% was reached. For evaluation purposes, models were saved at every 5% step in sparsity. Note that regularization term was defined on individual weights, not groups of weights. The additional hyperparameters are listed in Table 2 in Appendix A.

3.1 Sparsity-Robustness tradeoff

To distinguish the effect of pruning on test accuracy and robustness, I evaluated the pruned models on the original CIFAR-10 test set first. Figure 2 shows how accuracy changes during the compression process at different sparsity levels. To present the central tendency as well, the mean and standard deviation of the test accuracy across 5 different random seeds are plotted. However, due to the random dynamics of the regularization process, the sparsity levels are not identical after the corresponding pruning steps across different runs. Therefore, statistics were computed by assigning the models to predetermined sparsity anchor points and averaging the accuracies of the models which were mapped to the same anchor point. Please note that at anchor point of 45% sparsity, only one model was mapped across all runs, thus the standard deviation is zero.

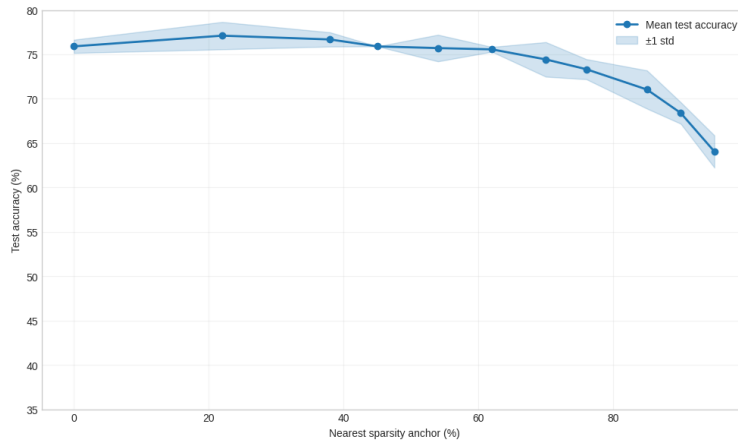


Figure 2: Test accuracy of pruned ResNet20 on CIFAR-10 dataset.

Although this plot does not quantify robustness, we can see how the model is able to maintain its performance below a sparsity level of 60%. When pruned more extensively, the accuracy starts to degrade approximately linearly with the sparsity level.

Figure 3 shows the test accuracies of the pruned models on the CIFAR-10-C dataset, averaged by corruption category across all severity level. The makeup of the categories is described in Appendix A. Note that the same anchor point aggregation methods was applied as well to compute the statistics based on the different runs.

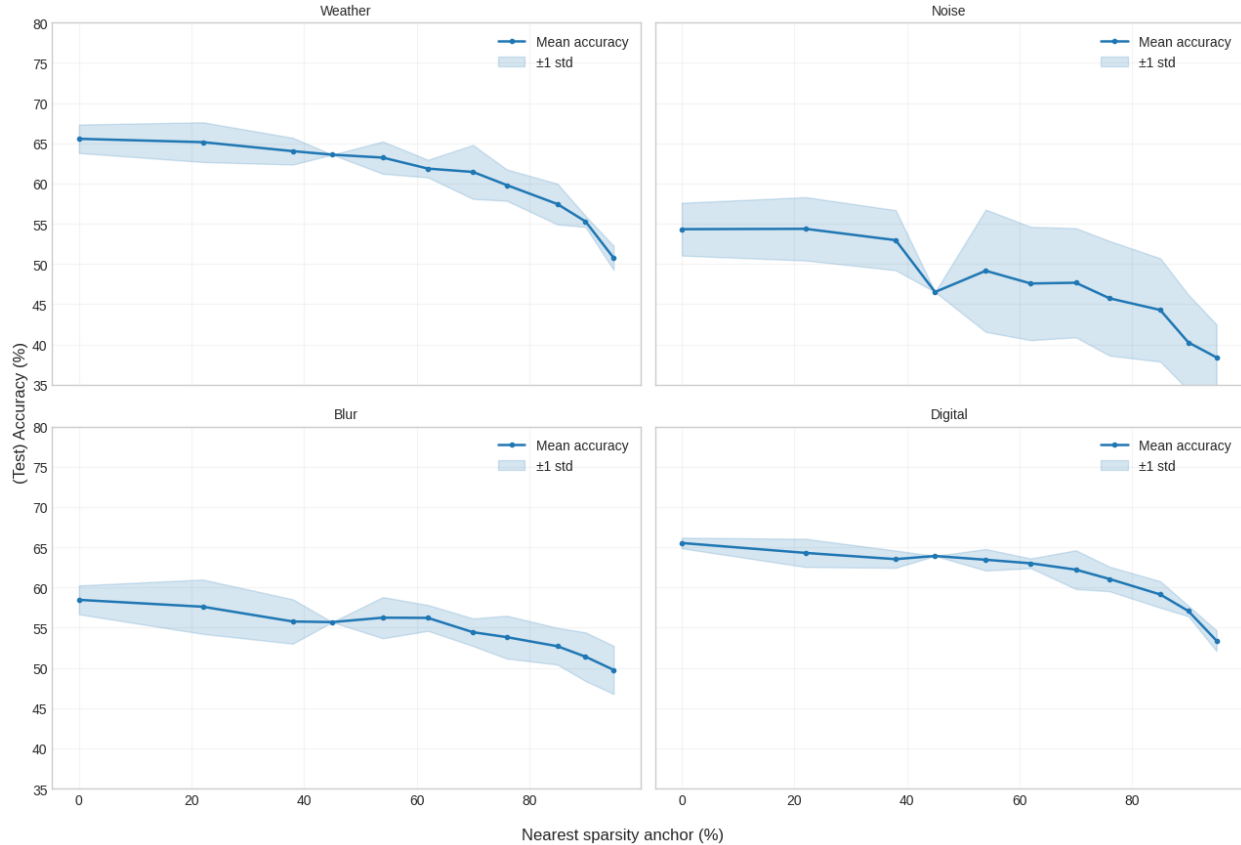


Figure 3: Test accuracy of pruned ResNet20 on CIFAR-10-C dataset, averaged by corruption category across 5 severity levels.

Not surprisingly, the performance of all models were degraded by at least 10% regardless of the corruption category or the sparsity level. However, as the shape of curves corresponding to all four categories are similar to the one on the original test set - resembling a nearly flat line until 60% sparsity and then a linear decrease -, we can infer that pruning does minimal harm to the robustness of the model against corruptions present in the CIFAR-10-C dataset. In other words, the pruned models are not more vulnerable to these corruptions than their dense counterpart.

Regarding the outlier standard deviation of accuracies on noise corruptions, we may conclude that the robustness of the model to high-frequency perturbations (i.e. gaussian noise, shot noise, etc.) is more sensitive to the the initial random initialization. However, this is not due to the pruning process, as the standard deviation appears to be similar across all sparsity levels. Therefore, the sensitivity of the model’s robustness to high-frequency perturbations is likely an inherent property of the architecture, rather than a consequence of pruning.

Apart from performance on corrupted images, we can evaluate the robustness of the pruned models by examining the distance between decision boundaries. As mentioned in section 1.2, the norm of the minimal perturbation found by an adversarial attack is a good proxy for this distance. The utilized attack is the Fast Adaptive Boundary attack, and the output of the attack will be referred to as FAB norm. Figure 4 visualizes the change in the

norms as the sparsity level increases during the pruning process. The same anchor point aggregation method was applied to compute the statistics based on the different runs. The FAB algorithm was run for 10 consecutive steps.

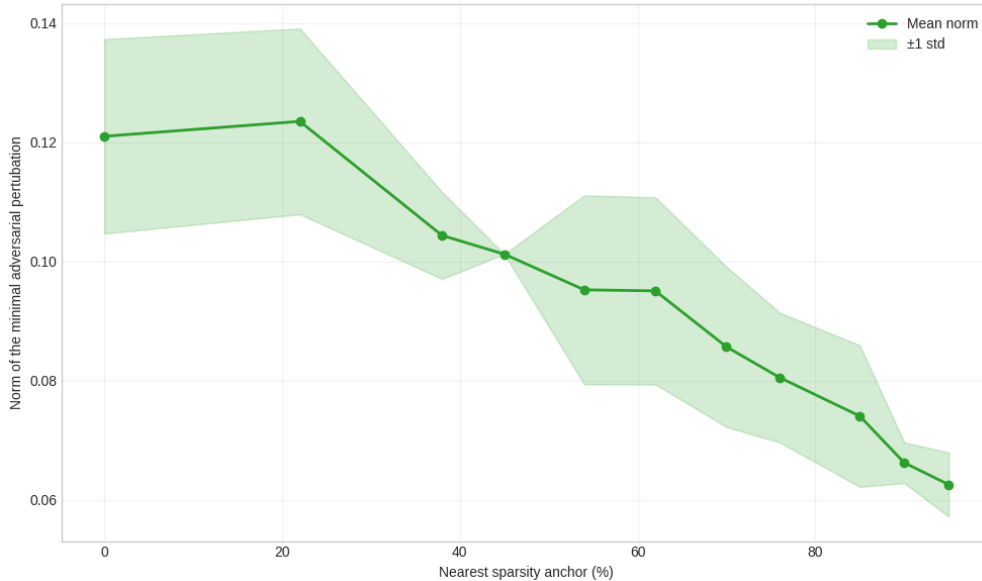


Figure 4: Adversarial robustness of pruned ResNet20 on CIFAR-10 dataset, measured by the norm of the minimal perturbation found by FAB attack.

Unlike the progression of test accuracies, the FAB norm starts to decrease earlier, after the 22% sparsity anchor, and it decreases approximately linearly afterwards. This indicates that the pruning process slowly but surely compresses the average margin to the decision boundary, thus making the model more vulnerable to small perturbations in its input. As the FAB attack is an iterative heuristic which finds approximations of the minimal perturbations, the high standard deviation may be reduced by increasing the number of steps the FAB algorithm can take.

3.2 Sensitivity analysis

To get more insight into how pruning affects OOD robustness, I utilized the Fourier sensitivity method described in section 2.3. Figure 5 presents two rows; the first row includes the Fourier heatmaps of the original dense model and five selected pruned models at different sparsity levels. The second row visualizes how pruning changes the baseline sensitivity of the dense model by showing the difference between the heatmaps of the latter and the pruned models. Note that the heatmap does not show the error rate itself, but the normalized difference between the error rate on the perturbed test set and the error rate on the original, clean test set. Therefore, brighter colors correspond to higher sensitivity to the corresponding Fourier basis perturbation. The reason behind plotting the normalized error difference instead of the errors on the perturbed test sets was due to the continuous decrease of test accuracy of pruned models on the clean test set. The aim of the plot was to visualize how the pruning process changes the sensitivity to distortions with specific frequencies and orientation, neglecting

the trivial overall sensitivity increase it causes. Normalizing them to be on the same scale also serves this purpose. The heatmap was calculated based on only one pruning run and its corresponding models with a L_2 perturbation budget of $\epsilon = 2.4$.

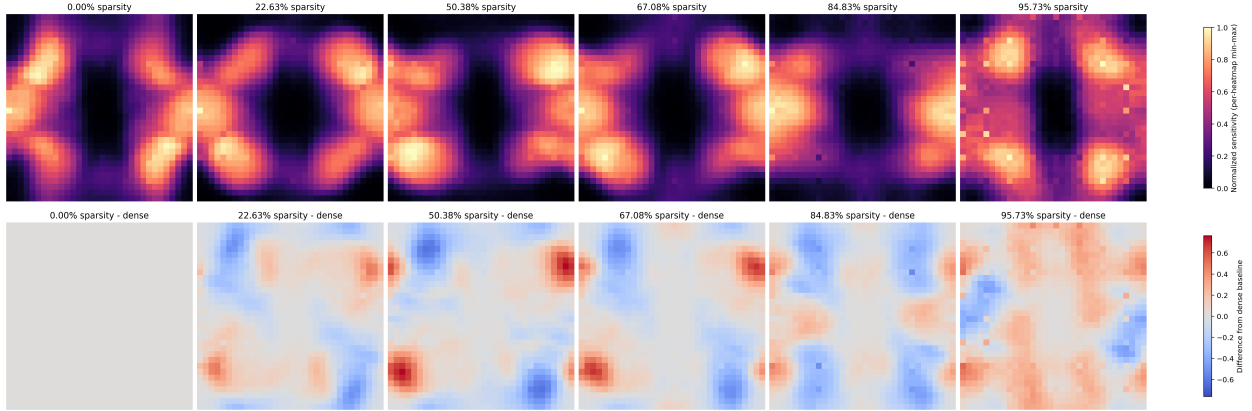


Figure 5: Sensitivity of pruned ResNet20 on CIFAR-10 dataset, measured by the norm of the minimal perturbation found by FAB attack.

By comparing the heatmaps of the first row, we can see that until the sparsity level of 84%, the two bright blobs with a half-moon shape remain relatively unchanged, although they appear to straighten out as sparsity increases. This indicates that the model does not lose its robustness to low-frequency perturbations due to pruning, as the center of the heatmap remains dark. Moreover, we can interpret the bright blobs losing their curvature as pruning degrades the model’s robustness to horizontal pixel shifts, while maintaining its robustness to vertical pixel shift. Based on the fourth image, the pruned model with sparsity level of 84.53% appears the most robust compared to the dense model. On the other hand, pruning beyond this level seems to cause a significant increase in sensitivity to all high-frequency perturbations corresponding to the bright circle in the heatmap. The last figures in the second row also indicate this sudden rise in sensitivity in general.

Apart from the pruned model at extreme sparsity, we might conclude that pruning preserved the overall shape of the Fourier sensitivity heatmap, thus maintaining the relative sensitivity to perturbations with different frequencies and orientations.

4 Summary

To sum up, the presented methods for measuring the impact of pruning on OOD and adversarial robustness of a neural network showed that the proposed gradual magnitude-based pruning strategy does not reshape the model’s robustness features. Evaluation on the CIFAR-10-C dataset indicated that the pruned models did not get more vulnerable to common corruptions than the original dense model. Furthermore, when examining the progression of the Fourier sensitivity heatmaps during the pruning process, no significant change could be detected in the model’s robustness to perturbations of different frequencies and orientations, except at extreme sparsity levels. However, the distance to the decision boundary - approximated by the norm of the minimal adversarial perturbation found by the

FAB attack - gradually decreased as sparsity levels increased. Based on these results, we hypothesize that our pruning process only decreases robustness to adversarial inputs, while maintains the OOD robustness of the model.

While the proposed pruning method did not worsen the OOD robustness, research suggests that model compression techniques like pruning can be enhanced to contribute to robustness targets as well during training. Therefore, future work is needed to find modifications of this specific pruning method to improve upon robustness, accuracy and compactness simultaneously.

References

- [1] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” *CoRR*, vol. abs/1905.11946, 2019.
- [2] R. Singh and S. S. Gill, “Edge ai: A survey,” *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023.
- [3] H. Cheng, M. Zhang, and J. Q. Shi, “A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 10558–10578, 2024.
- [4] K. Kireev, M. Andriushchenko, and N. Flammarion, “On the effectiveness of adversarial training against common corruptions,” *CoRR*, vol. abs/2103.02325, 2021.
- [5] J. Diffenderfer, B. R. Bartoldson, S. Chaganti, J. Zhang, and B. Kailkhura, “A winning hand: Compressing deep networks can improve out-of-distribution robustness,” *CoRR*, vol. abs/2106.09129, 2021.
- [6] J. O. Neill, “An overview of neural network compression,” 2020.
- [7] Z. Shen, J. Liu, Y. He, X. Zhang, R. Xu, H. Yu, and P. Cui, “Towards out-of-distribution generalization: A survey,” *CoRR*, vol. abs/2108.13624, 2021.
- [8] XU, Keyizhi, LU, Yajuan, WANG, Zhongyuan, and LIANG, Chao, “A survey of adversarial examples in computer vision: Attack, defense, and beyond,” *Wuhan Univ. J. Nat. Sci.*, vol. 30, no. 1, pp. 1–20, 2025.
- [9] A. Krizhevsky, “Learning multiple layers of features from tiny images,” technical report, University of Toronto, 2009.
- [10] D. Hendrycks and T. G. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” *CoRR*, vol. abs/1903.12261, 2019.
- [11] F. Croce and M. Hein, “Minimally distorted adversarial examples with a fast adaptive boundary attack,” *CoRR*, vol. abs/1907.02044, 2019.
- [12] D. Yin, R. G. Lopes, J. Shlens, E. D. Cubuk, and J. Gilmer, “A fourier perspective on model robustness in computer vision,” *CoRR*, vol. abs/1906.08988, 2019.

- [13] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” *CoRR*, vol. abs/1512.03385, 2015.
- [14] E. J. Candes, M. B. Wakin, and S. P. Boyd, “Enhancing sparsity by reweighted l1 minimization,” 2007.

A Appendix

Table 2: Hyperparameter Configuration for Training

Parameter	Value
Optimizer	Adam
Learning Rate	1×10^{-3}
Batch Size	64
Early Stopping Patience	8
Loss Function	Cross-Entropy
Activation	ReLU
Weight initialization	Uniform Xavier